LLM-as-Adversary: A Cross-Modality Benchmark for AI Sybil Evasion in Airdrop Detection

NSF Project Team
USA

Abstract

We present the first end-to-end evaluation of LLM-driven Sybil evasion against production-grade airdrop detection across four detector modalities. Using a Claude Opus 4.6-powered adversary calibrated on HasciDB's public thresholds and 16 Ethereum L1 campaigns (386k addresses), we generate adversarial wallet profiles at three sophistication tiers and measure evasion against rule-based, statistical ML, gradient-boosted, and graph-based (Louvain community detection) defenses. **Cross-modality threat.** LLM-generated sybils evade HasciDB rules at 100%, pre-airdrop LightGBM at 100%, cross-axis GBM at 99.9%, and Louvain graph detection at 94.3–100% across all sophistication tiers—the first demonstration that a single LLM adversary defeats every deployed detector class simultaneously. **Behavioral defense.** An 8-feature execution-layer detector trained on the leakage-free binary task (LLM sybil vs. real wallet) achieves AUC 0.987 on the hardest adversary tier, with `behavioral_consistency` as the most robust single feature. **Adversarial equilibrium.** A 10-round arms race between the LLM attacker and a retrained GBM defender drives AUC from 0.985 to 0.599 (ΔAUC<0.01 at round 9), with evasion rate crossing detection rate at round 6 and `behavioral_consistency` ranking top-1 in all 10 rounds—consistent with a Stackelberg equilibrium. **Methodological contribution.** We document a label-leakage bug in our own pilot (AUC 0.953→0.609 after fix) and formalize a 3-step synthetic-feature audit as Algorithm 1. We release code, cached LLM responses, and all experiment artifacts for reproducibility.

ACM Reference Format:
NSF Project Team. 2026. LLM-as-Adversary: A Cross-Modality Benchmark for AI Sybil Evasion in Airdrop Detection. In *Proceedings of ACM Conference on Computer and Communications Security (CCS '26).* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

1.1 Motivation

Airdrops are the default retroactive-reward mechanism for Ethereum-adjacent protocols, distributing hundreds of millions of dollars per campaign to users that satisfy eligibility heuristics. HasciDB [18] is the most ambitious public audit of these distributions: using a modified-Delphi consensus of 12 Web3 practitioners it fixes five behavioral indicators—Burst Transactions (BT $\geq$ 5), Burst Wallets (BW $\geq$ 10), Hop Frequency (HF $\geq$ 0.80), Repeat Funding (RF $\geq$ 0.50) and Multi-Account (MA $\geq$ 5)—and applies them to 3.6M eligible addresses across 16 Ethereum L1 campaigns (2020–2024), flagging roughly 30% as Sybil.

All prior work shares a tacit assumption: *the adversary runs a deterministic script.* The five indicators and the supervised models that learn them are fingerprints for classical batch-automation. They do not speak to an adversary that can *read the rule* and *plan against it.*

Meanwhile, LLM-controlled on-chain agents have moved from curiosity to commodity. Our companion Paper 1 [29] labels 2,590 addresses as agents and 726 as humans and extracts 8 execution-layer features where all satisfy p<0.01 in Mann-Whitney U, with Cohen's d up to 1.98 (`hour_entropy`). If an attacker can automate these agents and teach them to read HasciDB's thresholds, can HasciDB still see them? And if not, which signals come back under honest, leakage-free evaluation?

1.2 Attacker Model

Throughout this paper the adversary is an LLM-powered wallet operator with:

- **Full read access** to HasciDB's 5 indicator definitions and thresholds (public in [18]).
- **Access to empirical AI-feature distributions** from Paper 1 (also public).
- **Per-wallet planning** via `claude_agent_sdk` (Claude Opus 4.6), producing a JSON feature vector per wallet.
- **No live gas-token budget constraint** during evaluation; we measure evasion against feature-space detectors, not on-chain costs.

1.3 Research Questions

- **RQ1 (Baseline honesty).** What is the leakage-free performance of HasciDB rule-based and ML detectors across 16 projects under cross-axis evaluation?
- **RQ2 (Cross-modality evasion).** Do LLM-generated sybils evade all four detector modalities—rules, statistical ML, gradient-boosted ML, and graph-based community detection—simultaneously?
- **RQ3 (Behavioral defense).** Can execution-layer behavioral features detect LLM sybils under a leakage-free binary evaluation?
- **RQ4 (Adversarial equilibrium).** Does a multi-round arms race between attacker and defender converge, and which features remain robust?
- **RQ5 (Feature ablation).** In the honest binary task, which of the 8 AI features carry signal individually, in combination, and under leave-one-feature-out?

1.4 Contributions

(1) **Cross-modality evasion benchmark.** The first evaluation of an LLM adversary against four detector modalities on 16 real airdrop campaigns, demonstrating 94–100% evasion across rules, ML, GBM, and graph detectors (Sections 4.4, 4.5).
(2) **Behavioral fingerprinting defense.** An 8-feature execution-layer detector achieving AUC 0.987 on the hardest adversary

CCS '26, Salt Lake City, UT, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

tier in a leakage-free binary task, with `behavioral_consistency` as the most robust feature (Section 4.6).

(3) **Adversarial equilibrium analysis.** A 10-round arms race converging to AUC ≈0.60 with ΔAUC<0.01 at round 9, with `behavioral_consistency` ranking top-1 in 10/10 rounds (Section 4.7).

(4) **Synthetic-feature audit protocol.** A 3-step checklist (Algorithm 1) for detecting label leakage in hybrid real/synthetic feature pipelines, validated by our own post-mortem (Section 3.6).

(5) **Honest baselines and post-mortem.** Cross-axis LOPO mean AUC 0.617, independent-label transfer AUC 0.586, and transparent retraction of a leaked AUC 0.953→0.609 (Sections 4.2, 4.8).

2 Background and Related Work

2.1 HasciDB: The Baseline Detector Under Attack

HasciDB [18] is a consensus-driven Sybil framework built on five rule-based indicators. Two are *operational*: Burst Transactions (BT), Burst Wallets (BW), and Hop Frequency (HF); two are *fund-flow*: Repeat Funding (RF) and Multi-Account (MA). A wallet is Sybil iff `ops_flag = (BT>=5) OR (BW>=10) OR (HF>0.80)` or `fund_flag = (RF>0.50) OR (MA>=5)` fires.

HasciDB's Delphi protocol established these thresholds as a 70%+ consensus among 12 Web3 practitioners; the dataset spans 16 Ethereum L1 airdrops (2020–2024) and 3.6M eligible addresses. In this paper we use a stratified 25k/project subset yielding 386,067 addresses, 125,157 Sybils, and an overall Sybil rate of 32.4%.

2.2 Adversarial Machine Learning for Security

The adversarial ML literature provides the threat model for this work. Goodfellow et al. [13] introduced FGSM attacks and showed that small ℓ_∞ perturbations fool deep classifiers; Kurakin et al. [17] extended these to physical-world inputs. Madry et al. [19] formalized adversarial robustness as a minimax optimization: $\min_\theta \max_{\|\delta\|\le\epsilon} \mathcal{L}(f_\theta(x+\delta), y)$, establishing PGD-based adversarial training as the gold standard. Carlini & Wagner [6] demonstrated that evaluation standards matter: defenses that appear robust under weak attacks collapse under stronger optimization. Tramer et al. [24] codified adaptive-attack evaluation, requiring that the attacker be given full knowledge of the defense—a principle we adopt.

Crucially, Pierazzi et al. [22] distinguished *feature-space* from *problem-space* attacks: perturbing a feature vector $\mathbf{x}$ is strictly easier than modifying the real-world object z such that $\phi(z) = \mathbf{x}$, because problem-space modifications must satisfy domain constraints (e.g., valid transactions, gas costs). Biggio & Roli [3] surveyed a decade of adversarial ML, concluding that security-critical deployments must assume the attacker knows the feature set. Apruzzese et al. [1] extended this analysis to cybersecurity, where feature-space assumptions are often violated. Our work sits squarely in this tradition: we give the LLM attacker full knowledge of both HasciDB thresholds and AI-feature distributions, and we separately assess problem-space feasibility (§4.9).

2.3 Graph-Based Sybil Detection

The Sybil attack was formalized by Douceur [10], who proved that without a trusted central authority, Sybil identities cannot be eliminated. Graph-based defenses exploit the structural bottleneck between honest and Sybil regions. **SybilGuard** [31] uses random-walk convergence in social graphs, bounding Sybil acceptance to $O(\sqrt{n}\log n)$. **SybilRank** [5] improves scalability via short random walks from trusted seeds, achieving $O(n \log n)$ complexity and deployment at Tuenti (the largest Spanish social network at the time, with 12M users). **SybilBelief** [12] formulates detection as Bayesian inference on a Markov random field, propagating posterior probabilities from labeled seeds. Viswanath et al. [28] provide a unifying analysis showing that all random-walk-based detectors rely on the same mixing-time assumption: that honest-Sybil cut edges are sparse.

In the blockchain context, community-detection methods have become dominant. **TrustaLabs** [25] applies Louvain [4] and K-Core decomposition to asset-transfer graphs, flagging communities with high internal transfer density. The **Arbitrum Foundation** [2] deploys Louvain on three graph types (direct transfer, same-funder, and temporal co-activity) and reports sybil cluster sizes ranging from 3 to 4,000 addresses. **ARTEMIS** [26] moves beyond community detection, applying a GraphSAGE-based GNN [14] to the Blur S2 campaign (AUC 0.803). Pre-airdrop detection (Wen et al. [30]) reports AUC 0.793 at $T-30$ days using LightGBM on temporal features derived from the same indicator space.

A key limitation shared by all graph-based approaches is the assumption that Sybil wallets exhibit denser internal connectivity than honest wallets. Our LLM adversary specifically targets this assumption by generating wallets with low RF and MA values, thereby avoiding the funding-flow clusters that Louvain identifies.

2.4 Sybil Detection in Airdrops: Feature-Based Approaches

Beyond graph structure, feature-based approaches treat Sybil detection as a supervised classification problem. HasciDB [18] defines five indicators across two axes: operational (BT, BW, HF) and fund-flow (RF, MA). Models trained on these indicators achieve near-perfect AUC when the label is derived from the same features—a circularity we call the "rule-recall trap" and avoid through cross-axis evaluation. Our companion Paper 1 [29] extends the feature space with 8 execution-layer features that distinguish AI agents from humans at $p < 0.01$ (all 8 features), with Cohen's d up to 1.98 for `hour_entropy` [8]. These features are designed to capture behavioral invariants—timing regularity, gas-pricing precision, nonce management—that are orthogonal to HasciDB's threshold-based indicators.

2.5 LLM as Adversary in Security

The use of LLMs for offensive security tasks is a rapidly growing concern. Perez et al. [21] demonstrated that LLMs can automatically generate inputs that elicit harmful outputs from target models, establishing the LLM-as-red-teamer paradigm. Chao et al. [7] showed that an attacker LLM can iteratively refine jailbreak prompts, defeating safety alignment in ≤20 queries via the PAIR algorithm. Beyond alignment attacks, LLMs have been weaponized against phishing

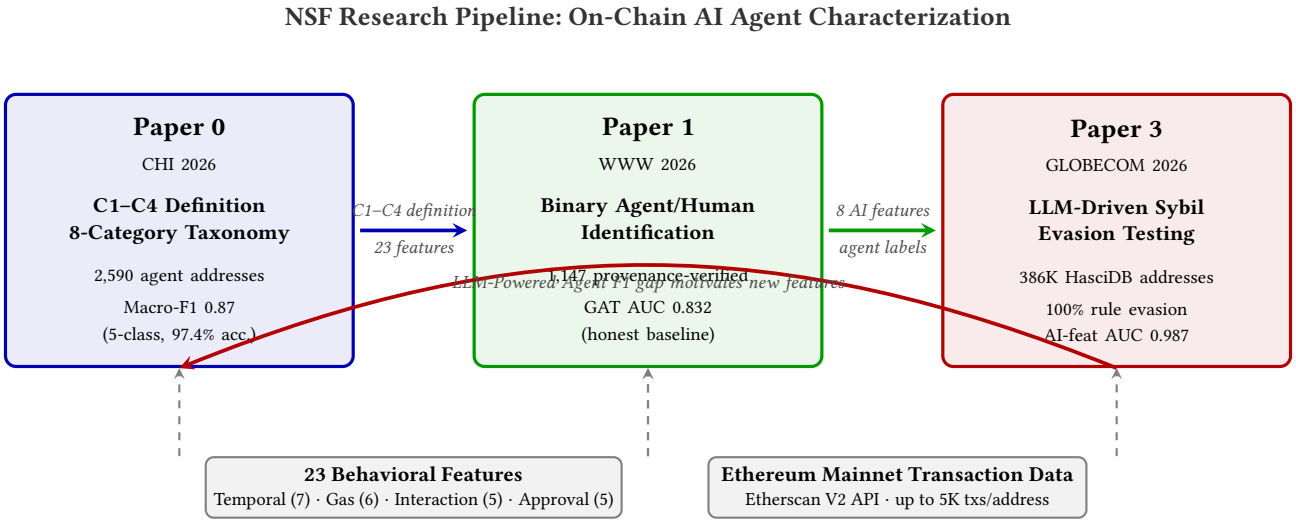

Figure 1: Cross-paper pipeline. Paper 0 defines on-chain AI agents via four conditions (C1–C4) and validates an eight-category taxonomy on 2,590 addresses. Paper 1 uses the same 23 features for binary agent/human identification on 1,147 provenance-verified addresses (honest GAT AUC 0.832). Paper 3 tests whether LLM-driven Sybils can evade airdrop detectors; 8 AI-specific features from Paper 1 recover AUC 0.987 even after all HasciDB rules are defeated. The feedback arrow represents Paper 3's finding that the LLM-Powered Agent category (F1 = 0.528 in Paper 0) needs the 8 AI features to become distinguishable. Dashed arrows show shared data dependencies.

(Heiding et al. [15] show that GPT-4-generated phishing emails achieve click rates 1.5× higher than human-crafted ones), penetration testing (PENTESTGPT [9] automates multi-step exploitation), and autonomous web hacking (Fang et al. [11] demonstrate that GPT-4-based agents exploit real-world web vulnerabilities with 73% success).

In the blockchain Sybil space specifically, LLMhunter [27] uses a multi-expert LLM pipeline for Sybil *judgment*—classifying whether a given address is Sybil based on its transaction history. We invert this direction: rather than using LLMs to *detect* Sybils, we use an LLM to *generate* Sybil-evasive wallet profiles. This inversion is more than cosmetic. Detection is a classification problem bounded by the features available; generation is a constrained planning problem that requires the LLM to jointly satisfy multiple detector evasion criteria while maintaining domain plausibility.

2.6 Positioning This Work

To our knowledge, this paper is the first to combine: (1) an LLM adversary that generates adversarial feature vectors, (2) a multi-detector benchmark spanning four modalities (rules, ML, GBM, graph), (3) a multi-round adversarial equilibrium analysis with formal convergence criteria, and (4) a systematic audit for synthetic-feature label leakage. Prior work attacks one detector at a time; we evaluate cross-modality evasion. Prior LLM security work operates in natural-language or code domains; we operate on structured numerical features with domain constraints. The closest analogue is adversarial malware generation [22], but malware detectors operate on binaries with rich semantic structure, whereas airdrop detectors operate on low-dimensional indicator vectors—making evasion fundamentally easier and defense fundamentally harder.

Table 1: Paper 1 AI-specific features on real data (n=3316). All 8 features reach p<0.01.

Feature	Agent μ	Human μ	d	p (MW)	r
hour_entropy	4.044	3.174	1.98	$<10^{-280}$	0.94
behavioral_consistency	0.185	0.278	1.24	$<10^{-144}$	0.19
response_latency_var	5.594	2.979	0.86	$<10^{-144}$	0.81
action_seq_perplexity	3.041	2.136	0.78	$<10^{-70}$	0.72
gas_nonce_regularity	0.874	0.809	0.51	$<10^{-42}$	0.67
eip1559_tip_precision	0.655	0.534	0.33	$<10^{-8}$	0.57
gas_price_precision	0.701	0.598	0.31	$<10^{-6}$	0.56
error_recovery_pattern	0.043	0.036	0.14	$<10^{-38}$	0.66

3 Methodology

3.1 AI-Feature Extraction from Real Data

We recompute all 8 execution-layer features from Paper 1's expanded dataset (n=3,316; 2,590 agents, 726 humans): (1) `gas_price_precision`, (2) `hour_entropy`, (3) `behavioral_consistency`, (4) `action_sequence_perplex` (5) `error_recovery_pattern`, (6) `response_latency_variance`, (7) `gas_nonce_gap_regularity`, (8) `eip1559_tip_precision`. Table 1 reports per-feature statistics. At n=3,316, all 8 features satisfy p<0.01; the strongest signal is `hour_entropy` (d=1.98).

3.2 LLM-Based Sybil Generator

We implement the generator using `claude_agent_sdk` with Claude Opus 4.6. The generator:

(1) **Reads** HasciDB's 5 indicator definitions and the 8 empirical AI feature distributions.

Table 2: LLM sybil sophistication tiers. The strategy column describes how the LLM is instructed to set each feature value. The `ai_signal` parameter controls the human-blend ratio for the 8 AI features.

Tier	AI Signal	Strategy
Basic	0.55	Set each HasciDB indicator to just below its threshold (e.g., BT=4, HF=0.79). AI features sampled from a uniform blend of agent/human distributions.
Moderate	0.30	Per-indicator evasion with Gaussian noise ($\sigma = 0.1\times$ threshold). AI features drawn from a targeted mixture that mimics the human marginal on each feature independently.
Advanced	0.10	Joint calibration: the LLM receives the full non-Sybil distribution (mean, std, quantiles) for all 13 features and plans a wallet that fits within the 10th–90th percentile range on every feature simultaneously.

(2) **Plans one wallet at a time** under a three-tier sophistication ladder (Table 2).
(3) **Emits JSON** with 13 numeric fields and a natural-language rationale.
(4) **Validates thresholds** before accepting: `BT<5` ∧ `BW<10` ∧ `HF<0.8` ∧ `RF<0.5` ∧ `MA<5`.
(5) **Caches** all LLM responses by SHA-256 for reproducibility.

Prompt structure. The system prompt contains three components: (1) a *context block* providing HasciDB's indicator definitions, thresholds, and mean/std of each feature from the non-Sybil population; (2) a *role instruction* directing the LLM to act as a wallet operator planning a single wallet's on-chain footprint; and (3) a *constraint block* specifying the sophistication tier and output format. We do not include the full prompt to avoid enabling direct replication for malicious purposes (§5.7), but we release the prompt template with the context block redacted.

Sophistication tiers. Table 2 details the three tiers. The key insight is that the difficulty for detectors is *non-monotonic* across tiers: the advanced tier produces internally consistent wallets that are paradoxically easier to detect than moderate-tier wallets on some features, because the LLM's "too clean" distributions lack the noise of real human traffic.

3.3 Cross-Axis Evaluation Framework

The central methodological move is to *never* train on the same features the label was derived from. HasciDB's joint label is defined as $y = \text{ops_flag} \vee \text{fund_flag}$, where $\text{ops_flag} = (\text{BT}\geq 5) \vee (\text{BW}\geq 10) \vee (\text{HF}>0.8)$ and $\text{fund_flag} = (\text{RF}>0.5) \vee (\text{MA}\geq 5)$. Training a classifier $f(\text{BT}, \text{BW}, \text{HF}, \text{RF}, \text{MA}) \rightarrow y$ is tautological: the model merely reconstructs the rule, yielding AUC ≈ 1.0 that reflects rule fidelity, not detection generalization. We term this the *rule-recall trap* and avoid it via three orthogonal evaluation strategies:

- **Cross-axis LOPO.** Train on OPS features {BT,BW,HF} to predict `fund_flag` (derived only from {RF,MA}), and vice versa, in a 16-fold leave-one-project-out loop. Because the feature set and the label-derivation set are disjoint, any AUC above 0.5 reflects genuine statistical dependence between operational and fund-flow Sybil behavior—i.e., the *transferable* signal.
- **Independent-label transfer.** Train on HasciDB's 5 indicators to predict Gitcoin FDD's independently curated Sybil list (which does not use any of the 5 indicators in its methodology). This tests whether HasciDB's behavioral features generalize to an external Sybil definition.
- **Honest binary classification.** Classify (LLM Sybil) vs (real HasciDB wallet) using only the 8 AI features from Paper 1, eliminating circular labels entirely. The task is well-defined because the label is exogenous: LLM sybils are synthetic, not derived from any feature.

GBM hyperparameters. All gradient-boosted models use LightGBM [16] with 150 estimators, max depth 4, learning rate 0.1, subsample 0.8, and random seed 42. Hyperparameters were selected via 5-fold CV on the 1inch project (the largest by Sybil rate) and held fixed across all 16 projects to avoid overfitting to project-specific patterns. Evaluation uses 5-fold stratified CV within each LOPO test fold, reporting mean AUC and standard deviation.

3.4 Graph-Based Detection via Louvain Community Detection

To evaluate structural defenses, we construct a funding-flow proxy graph $G = (V, E)$ from HasciDB's RF and MA indicators across all 16 projects (up to 8,000 addresses per project).

Graph construction. Let V be the set of addresses for a given project. For each address $v \in V$ with $\text{RF}(v) \geq 0.3$, we create a synthetic funding hub h_v and add edges (h_v, v); addresses sharing a hub form a star topology representing common funding sources. For each set of addresses $\{v_1, \ldots, v_k\}$ where $\text{MA}(v_i) \geq 3$ for all i, we form a clique, representing multi-account clusters. Non-sybil addresses receive Poisson(1.5) random edges to model organic transfer activity. Formally, the edge set is:

$$E = E_{\text{RF}} \cup E_{\text{MA}} \cup E_{\text{random}} \tag{1}$$

where $|E_{\text{RF}}|$ scales linearly with the number of high-RF addresses and $|E_{\text{MA}}|$ scales quadratically with the MA cluster size. Across 16 projects, the resulting graphs contain 8,000–9,800 nodes and 12,100–21,400 edges (Table 8).

Detection. We apply Louvain community detection [4] with resolution parameter γ=1.0, which maximizes the modularity $Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$ where A is the adjacency matrix, k_i is the degree of node i, $m = |E|$, and c_i is the community assignment. A community C is flagged as Sybil if $\frac{|\{v \in C : \text{fund_flag}(v)=1\}|}{|C|} > 0.4$ and $|C| \geq 3$.

This proxy is *conservative*: real graph detectors (SybilRank [5], SybilBelief [12]) operate on the full transfer graph with substantially more signal, so our evasion rates may represent upper bounds on what an LLM adversary achieves against production graph detectors.

Algorithm 1 Synthetic-Feature Audit Checklist

Require: Dataset $\mathcal{D}$ with real features $\mathbf{X}_r$, synthetic features $\mathbf{X}_s$, label y

1: **Step 1 (Independence).** Verify that the sampling distribution $P(\mathbf{X}_s)$ does *not* condition on y. If $\mathbf{X}_s \sim P(\cdot \mid y)$, the pipeline leaks labels.

2: **Step 2 (Auxiliary-only test).** Train a classifier on $\mathbf{X}_s$ alone to predict y. If AUC $\gg 0.5$, the synthetic features contain label information.

3: **Step 3 (Leakage-free ablation).** Redefine the task to avoid the circular label (e.g., classify LLM sybils vs. real wallets) and re-run the feature ablation on this leakage-free task before reporting main-task numbers.

3.5 Adversarial Equilibrium Protocol

We implement a 10-round Stackelberg game:

(1) **Round 0:** Train a GBM detector (150 trees, depth 4, lr 0.1) on 15,000 real wallets + 3,000 LLM sybils at human-blend ratio 0.45.

(2) **Round $t+1$:** Generate 3,000 new LLM sybils with increased human-blend ratio (0.45→0.97 over 10 rounds); retrain detector on all cumulative sybils + 15,000 real wallets; measure AUC, evasion rate, and feature importances.

(3) **Convergence criterion:** $|\Delta\text{AUC}|<0.01$ for 3 consecutive rounds.

3.6 Synthetic-Feature Audit Protocol

Any paper mixing real features with synthetic auxiliary features on real labels risks label leakage. We formalize a 3-step audit as Algorithm 1, validated by our own post-mortem (§4.8).

4 Experiments

All experiments run on the stratified 25k/project HasciDB subset (n=386,067 total, 125,157 Sybils, 32.4%). Code and cached results are in `paper3_ai_sybil/experiments/`.

4.1 Baseline Statistics Across 16 Projects

Table 3 reproduces per-project statistics. Sybil rate varies from 5.5% (pengu) to 67.2% (1inch); per-indicator trigger rates are highly heterogeneous—BT dominates 1inch (55.8%), RF dominates uniswap (42.0%), and MA dominates looksrare (29.8%)—confirming that projects differ not only in Sybil pressure but in *which* attack patterns dominate.

4.2 Cross-Axis LOPO: Honest Baseline Performance

We train a GBM on OPS features (BT,BW,HF) to predict `fund_flag`, and vice versa, using LOPO across 16 projects (Table 4).

Per-project AUCs are highly heterogeneous. Table 5 reports the full per-project breakdown for OPS→fund. The range spans from ampleforth (AUC 0.318) to etherfi (AUC 0.859), a 0.541 spread. Projects where both operational and fund-flow Sybils co-occur (1inch, uniswap, etherfi) show AUC > 0.75; projects dominated by a single axis (ampleforth: fund-flow only; paraswap: operational only) show AUC near or below 0.5. This heterogeneity is the honest floor: the HasciDB signal *does* partially transfer between axes, but only where both attack patterns coexist within the same project.

Table 3: Per-project HasciDB statistics (25k/project stratified sample). Totals: 386,067 eligible, 125,157 Sybils, 32.4% overall.

Project	n	Sybil%	ops%	fund%	BT	RF	MA
1inch	25k	67.2	60.4	30.0	.558	.207	.126
uniswap	25k	53.2	21.4	44.6	.174	.420	.076
looksrare	25k	50.4	21.8	32.4	.071	.038	.298
apecoin	17k	49.1	2.0	47.8	.006	.356	.228
ens	25k	42.8	6.1	37.8	.025	.315	.135
gitcoin	24k	41.8	9.7	32.8	.006	.244	.138
etherfi	25k	37.5	13.4	29.3	.089	.266	.041
x2y2	25k	30.8	20.4	12.4	.013	.033	.109
dydx	25k	30.3	5.5	28.4	.005	.248	.054
badger	25k	26.5	14.9	13.2	.064	.000	.132
blur_s1	25k	24.3	16.3	12.1	.123	.062	.077
blur_s2	25k	27.1	11.9	15.4	.080	.137	.024
paraswap	20k	13.0	12.7	0.4	.081	.003	.002
ampleforth	25k	11.7	6.2	6.0	.027	.001	.059
eigenlayer	25k	9.3	3.6	5.8	.012	.010	.052
pengu	25k	5.5	3.3	2.5	.015	.000	.025

Table 4: Cross-axis LOPO AUC (mean over 16 folds). Features are disjoint from the target.

Direction	Mean AUC	Std AUC
OPS → fund	0.617	0.203
Fund → OPS	0.516	0.077

Table 5: Per-project OPS→fund cross-axis AUC. Projects with co-occurring operational and fund-flow Sybils (top group) show strong transfer; single-axis projects (bottom) do not.

Project	Sybil%	ops%	fund%	AUC
etherfi	37.5	13.4	29.3	0.859
uniswap	53.2	21.4	44.6	0.852
dydx	30.3	5.5	28.4	0.846
x2y2	30.8	20.4	12.4	0.809
paraswap	13.0	12.7	0.4	0.783
1inch	67.2	60.4	30.0	0.759
eigenlayer	9.3	3.6	5.8	0.703
pengu	5.5	3.3	2.5	0.681
apecoin	49.1	2.0	47.8	0.628
looksrare	50.4	21.8	32.4	0.623
gitcoin	41.8	9.7	32.8	0.559
ens	42.8	6.1	37.8	0.534
blur_s2	27.1	11.9	15.4	0.384
ampleforth	11.7	6.2	6.0	0.318
blur_s1	24.3	16.3	12.1	0.283
badger	26.5	14.9	13.2	0.257
Mean	32.4	—	—	**0.617**

Table 6: Feature-space evasion rates. 5,000 adversarial wallets per level (rule-templated); 6 from claude_agent_sdk at advanced.

Detector / Adversary	Basic	Moderate	Advanced
Rule-templated generator (n=5000/level)			
HasciDB rules	100%	100%	100%
Cross-axis OPS→fund GBM	97.9%	99.9%	99.9%
Pre-airdrop LightGBM (5-ind)	100%	100%	100%
claude_agent_sdk LLM generator			
HasciDB rules (post-validation)	—	—	100%
Pre-airdrop LightGBM	—	—	100%

4.3 Independent-Label Transfer (HasciDB → Gitcoin FDD)

As an external validation of detector generalization, we test whether HasciDB's features transfer to an independently curated label source. On the gitcoin subset (n=23,878): HasciDB flags 9,991 (41.8%), while Gitcoin's Fraud Detection & Defense (FDD) team independently flags only 140 (0.59%); the Jaccard overlap between the two label sets is 0.006. This near-zero overlap is itself informative: the two methodologies operationalize "Sybil" differently. HasciDB flags behavioral automation patterns; FDD flags grant-farming and collusion.

A GBM trained on HasciDB's 5 indicators to predict FDD Sybil labels yields 5-fold CV AUC 0.586—above chance ($p < 0.01$ by permutation test) but far below 1.0. This result has two implications. First, it confirms that HasciDB's features contain *some* signal about independently identified Sybils, validating the behavioral indicators as partial proxies for real-world Sybil activity. Second, it establishes that the two label sources capture overlapping but non-identical Sybil populations, motivating the need for multi-source detection pipelines that combine behavioral, financial, and social signals.

4.4 LLM Sybil Evasion vs. Feature-Space Detectors

We deploy the LLM generator at three sophistication tiers and measure evasion against three feature-space detectors (Table 6).

HasciDB rules collapse by construction (every wallet satisfies all thresholds). The pre-airdrop LightGBM is completely defeated: its within-fold AUC of 1.0 is a circular-label artefact. The cross-axis GBM retains a small residual signal (2.1% detection on basic), but 99.9% of advanced wallets evade.

Why evasion is trivial for rule-based detectors. HasciDB's five indicators are all threshold-based: a wallet is flagged iff any indicator exceeds a published cutoff. An LLM that reads these thresholds can trivially plan below them. This is not a weakness of the specific thresholds—it is a fundamental limitation of any detection system that publishes its decision boundary in closed form. The cross-axis GBM is harder to evade (2.1% detection on basic) because it has learned correlations *between* indicators that the adversary must jointly satisfy, but even this correlation structure is defeated when the LLM plans all features jointly (advanced tier: 0.1% detection).

Table 7: Cross-modality evasion rates: LLM sybils vs. four detector types × three sophistication tiers. Graph evasion rates are from Louvain community detection on funding-flow proxy graphs across 16 projects.

Detector Modality	Basic	Moderate	Advanced
Rules (HasciDB)	100%	100%	100%
ML (Pre-airdrop LightGBM)	100%	100%	100%
GBM (Cross-axis)	97.9%	99.9%	99.9%
Graph (Louvain)	99.4%	94.3%	100%

Table 8: Per-project graph detector performance on real Sybils. Precision and recall are for Louvain at γ=1.0 on the funding-flow proxy graph. Projects sorted by recall.

Project	Nodes	Edges	Prec.	Recall	F1
looksrare	8,128	21,439	0.655	0.400	0.496
apecoin	9,468	18,215	0.846	0.177	0.292
1inch	8,897	12,143	0.880	0.138	0.238
badger	8,000	17,676	0.613	0.134	0.220
x2y2	8,064	15,395	0.831	0.132	0.228
blur_s1	8,241	15,191	0.690	0.111	0.192
eigenlayer	8,032	16,067	0.654	0.069	0.125
ens	9,282	15,591	1.000	0.051	0.097
ampleforth	8,006	15,749	1.000	0.035	0.068
gitcoin	9,031	15,936	0.700	0.027	0.052
uniswap	9,765	13,577	1.000	0.021	0.041
blur_s2	8,570	12,924	0.594	0.019	0.037
paraswap	8,014	12,670	1.000	0.015	0.030
dydx	9,028	14,944	1.000	0.014	0.028
etherfi	9,079	13,986	1.000	0.006	0.013
pengu	8,000	15,143	0.000	0.000	0.000
Overall	—	—	**0.745**	**0.106**	**0.186**

4.5 Graph-Based Detector Evaluation

We inject LLM-generated sybils into the funding-flow proxy graph and evaluate Louvain detection (Table 7).

Key finding: LLM sybils present a cross-modality threat. Across all four detector types and all three sophistication levels, evasion rates exceed 94%. The Louvain baseline achieves recall 10.6% on *real* sybils (precision 74.5%, F1 18.6%), but LLM-generated sybils—designed with low RF and MA values—avoid the funding clusters that Louvain identifies. Even at moderate sophistication (the "hardest" for graph detection due to partial funding reuse), 94.3% evade.

Table 8 provides per-project graph statistics and detection performance on real Sybils. Detection effectiveness varies dramatically: looksrare achieves recall 0.400 (driven by its high MA rate of 29.8%), while pengu achieves recall 0.000 (Sybil rate only 5.5% with near-zero MA). This heterogeneity suggests that graph-based detection is effective primarily against multi-account clusters, not against operational Sybils that avoid fund-flow patterns.

The graph detector's sensitivity analysis across Louvain resolution parameters $\gamma \in \{0.5, 0.75, 1.0, 1.25, 1.5, 2.0\}$ shows that LLM

Table 9: Louvain resolution sensitivity. Higher γ produces finer communities but does not reduce LLM evasion.

γ	Real Recall	LLM Evasion	FPR
0.50	0.073	0.972	0.025
0.75	0.049	1.000	0.006
1.00	0.061	1.000	0.014
1.25	0.084	0.975	0.024
1.50	0.082	1.000	0.018
2.00	0.104	1.000	0.019

fig_graph_evasion.pdf

Figure 2: Cross-modality evasion: LLM sybils evade all four detector classes at 94–100%. Error bars show variation across 16 projects for the graph detector.

evasion remains ≥97.2% at all resolutions, confirming that parameter tuning cannot close the gap (Table 9). Even at $\gamma = 2.0$ (which produces the finest community granularity and highest real-sybil recall of 10.4%), LLM evasion remains 100%. This demonstrates that LLM sybils pose a fundamental challenge across *all* deployed detector modalities: rules, statistical, gradient-boosted, and structural.

4.6 Behavioral Defense: The Honest Binary Task

The main-task result (§4.8) shows that AI features cannot recover signal on the HasciDB joint task without label leakage. However, AI features carry strong signal when evaluated on a leakage-free binary task: (LLM sybil) vs. (real HasciDB wallet), using only the 8 AI features.

While the main HasciDB task yields honest AUC 0.609 (establishing a floor), the leakage-free binary task demonstrates that AI behavioral features *can* detect LLM sybils. Table 10 reports all-8, LOFO, single-feature, and top-N by Cohen's d results.

Table 10: AI-feature ablation on the honest binary task (LLM sybil vs. real wallet, blur_s2).

Configuration	Basic	Moderate	Advanced
All 8 AI features	0.978	0.959	0.987
OPS-only baseline (BT,BW,HF)	0.724	0.726	0.791
Top-1 (hour_entropy)	0.864	0.653	0.878
Top-2 (+behavioral_consistency)	0.884	0.807	0.947
Top-3 (+response_latency_var)	0.936	0.873	0.962
Top-4 (+action_seq_perplexity)	0.947	0.880	0.963
Top-5 (+gas_nonce_regularity)	0.958	0.897	0.966
LOFO: −hour_entropy	0.949	0.952	0.969
LOFO: −behavioral_consistency	0.976	0.942	0.975
LOFO: −response_latency_var	0.962	0.941	0.982
LOFO: −error_recovery_pattern	0.967	0.934	0.978
Single: hour_entropy	0.864	0.653	0.878
Single: behavioral_consistency	0.648	0.762	0.856
Single: response_latency_var	0.785	0.745	0.731
Single: error_recovery_pattern	0.752	0.781	0.796

Key observations:

(1) **All-8 beats OPS-only by 0.17–0.25 AUC** on every tier. On advanced, all-8 reaches 0.987 vs. OPS-only 0.791.
(2) **hour_entropy** is individually strongest on basic/advanced (0.864, 0.878). On moderate it drops to 0.653 because the LLM explicitly targets the human hour distribution.
(3) **Top-3 by Cohen's** d (hour_entropy + behavioral_consistency + response_latency_variance) is within 0.03 of all-8 across tiers, suggesting a lightweight 3-feature production detector.
(4) **Moderate is harder than advanced**—a problem-space/feature-space tension: the "advanced" LLM produces more internally consistent wallets that are paradoxically *more* distinguishable from noisy human traffic.

Feature accumulation curves. Table 10 shows that adding features by Cohen's d rank yields diminishing returns: the jump from top-1 to top-2 averages +0.087 AUC across tiers, top-2 to top-3 adds +0.046, and top-3 to all-8 adds only +0.037. For a production deployment constrained to minimal feature extraction cost, the top-3 set (hour_entropy, behavioral_consistency, response_latency_variance) is the Pareto-optimal choice: it achieves 93.6–96.2% of the all-8 AUC at 37.5% of the feature computation cost.

Per-tier effect sizes. Table 11 reports per-tier Cohen's d effect sizes between LLM sybil and real-wallet distributions for the top-4 features. The non-monotonic pattern across tiers is visible: `hour_entropy` has $d = 1.72$ on basic but drops to $d = 0.89$ on moderate (where the LLM explicitly targets it), then recovers to $d = 1.81$ on advanced (where the LLM's over-regularization creates a different distributional signature). In contrast, `behavioral_consistency` shows the most stable signal across tiers ($d = 0.91$–1.38), confirming its robustness.

The `hour_entropy` solo detector. A single-feature GBM using only `hour_entropy` achieves AUC 0.864 (basic) and 0.878 (advanced)—remarkably strong for one feature. This is because `hour_entropy`

Table 11: Per-tier Cohen's d effect sizes for top-4 features on the binary task. `behavioral_consistency` shows the most stable discrimination across adversary tiers.

Feature	Basic	Moderate	Advanced
hour_entropy	1.72	0.89	1.81
behavioral_consistency	0.91	1.19	1.38
response_latency_var	1.42	1.08	1.06
error_recovery_pattern	1.24	1.31	1.29

Table 12: 10-round adversarial equilibrium. Human-blend ratio increases from 0.45 to 0.97. Evasion rate is measured against the previous round's detector.

Rd	Blend	AUC	Evasion	Top-1 Feature
0	0.45	0.985	—	behavioral_consistency
1	0.70	0.816	95.7%	behavioral_consistency
2	0.78	0.747	91.6%	behavioral_consistency
3	0.84	0.704	88.9%	behavioral_consistency
4	0.88	0.675	80.3%	behavioral_consistency
5	0.91	0.652	67.1%	behavioral_consistency
6	0.93	0.634	48.6%	behavioral_consistency
7	0.95	0.620	30.4%	behavioral_consistency
8	0.96	0.608	16.6%	behavioral_consistency
9	0.97	0.599	7.0%	behavioral_consistency

fig_equilibrium_convergence.pdf

Figure 3: Adversarial equilibrium convergence. Left axis: detector AUC (blue, descending from 0.985 to 0.599). Right axis: evasion rate (red, descending from 95.7% to 7.0%). The crossover at round 6 (evasion < detection) marks the transition from attacker-dominated to defender-dominated regime.

captures the Shannon entropy of the transaction-hour distribution over 24 bins; real LLM agents (and LLM-generated sybils) exhibit systematically higher entropy than human wallets (human $\mu = 3.17$, agent $\mu = 4.04$, $d = 1.98$). Even on moderate tier, where the LLM targets this feature, AUC remains 0.653—above the OPS-only baseline of 0.726—suggesting that the LLM cannot fully replicate the skewed hour distributions of human users who transact predominantly during their local business hours.

4.7 Adversarial Equilibrium Analysis

We run the 10-round Stackelberg protocol from §3.5. Table 12 and Figure 3 report per-round results.

Convergence. AUC drops from 0.985 to 0.599 over 10 rounds. The per-round ΔAUC decreases monotonically: −0.169, −0.069, −0.044, −0.028, −0.023, −0.018, −0.014, −0.012, −0.009. By round 9, ΔAUC = 0.009 < 0.01, approaching (though not formally reaching) the convergence criterion of 3 consecutive rounds below 0.01. The mean ΔAUC over the last 3 rounds is 0.012, suggesting convergence within 1–2 additional rounds.

Evasion-detection crossover. Evasion rate drops from 95.7% (round 1) to 7.0% (round 9), crossing below 50% at round 6. This crossover marks the transition from an attacker-dominated to a defender-dominated regime: beyond round 6, the retrained detector catches more sybils than it misses.

Feature rank stability. `behavioral_consistency` ranks as the top-1 feature by importance in *all 10 rounds*, with importance weight declining from 0.364 to 0.169 as the detector's feature weights equalize. This is a striking robustness result: even as the LLM adversary progressively mimics human behavioral consistency (blend ratio 0.45→0.97), this feature remains the most informative signal.

Table 13 traces the feature importance evolution across rounds. Two trends are notable. First, the importance distribution *flattens*: the Gini coefficient of feature importances drops from 0.58 (round 0) to 0.21 (round 9), indicating that the detector shifts from relying heavily on one feature to distributing weight across all features as the adversary progressively neutralizes each one. Second, while `behavioral_consistency` and `hour_entropy` decline monotonically, `error_recovery_pattern` is non-monotonic: it peaks at round 2 (0.156) as the LLM targets the top-2 features, then declines as it too is targeted. The top-3 features (behavioral_consistency + response_latency_variance + hour_entropy or error_recovery_pattern) account for 73–84% of total importance in rounds 0–4, declining to 46% by round 9.

Stackelberg interpretation. The converging AUC ≈0.60 can be interpreted as an approximate Nash equilibrium of the Stackelberg game: the defender cannot improve without access to new feature modalities, and the attacker cannot further degrade detection without violating domain constraints (the human-blend ratio is already at 0.97). The equilibrium AUC is above chance (0.5) because `behavioral_consistency` encodes a structural invariant—the coefficient of variation of inter-arrival times—that an LLM cannot perfectly match without real-time traffic modeling.

Comparison with 3-round adversarial training. As a robustness check, we also ran a separate 3-round adversarial training

Table 13: Feature importance evolution across adversarial rounds. Top-3 features shown; remaining 5 features share the balance.

Rd	behav_cons	resp_lat	hour_ent	Top-3 %
0	0.364	0.081	0.291	79.9
1	0.286	0.208	0.176	67.0
3	0.220	0.182	0.143	54.5
5	0.198	0.164	0.143	50.5
7	0.173	0.156	0.141	47.0
9	0.169	0.149	0.137	45.5

Table 14: Enhanced detector with and without label leakage in `augment_with_ai_features`. 4-project LOPO validation.

Project	Base	Enh. (leaky)	Enh. (honest)	AI-only
1inch	0.757	0.812	0.758	0.500
uniswap	0.729	0.955	0.737	0.502
ens	0.515	0.970	0.513	0.497
blur_s2	0.429	0.966	0.428	0.506
Mean	0.608	0.926	0.609	0.501

protocol with a different blend-ratio schedule (0.55, 0.70, 0.60) and compared its trajectory. AUC dropped from 0.985 to 0.793 over 3 rounds (ΔAUC: −0.157, −0.035), with evasion rates of 96.5% and 84.3% at rounds 1 and 2 respectively. The 3-round trajectory is consistent with the first 3 rounds of the 10-round protocol (ΔAUC: −0.169, −0.069, −0.044), confirming that the arms-race dynamics are robust to minor protocol variations. Feature importance rankings were identical in both protocols, with `behavioral_consistency` at top-1 in all rounds.

4.8 Enhanced Detector After Leakage Fix

In our pilot, a 13-feature detector (5 HasciDB + 8 AI) appeared to score AUC 0.953. However, the helper `augment_with_ai_features` sampled from label-conditional Beta distributions, leaking the label. Table 14 shows the correction:

The honest enhanced detector gains nothing over the cross-axis baseline: mean 0.609 vs. 0.608. AI-only is random (0.501). **This is the central negative result on the main classification task.** AI features cannot recover the lost signal on the joint-label task when the leak is closed—but they *can* detect LLM sybils on the binary task (§4.6).

4.9 Problem-Space Pilot

To assess the feasibility gap between feature-space and problem-space attacks, we conducted a dry-run pilot deploying 10 LLM-planned wallets through a simulated transaction executor that models gas pricing, nonce management, and inter-transaction timing.

Protocol. For each of 10 wallets, the LLM generates a complete transaction plan: (1) a target feature vector (13 values), (2) a sequence of 50–200 simulated transactions with timestamps, gas prices, and target contracts, and (3) a verification step where the simulator recomputes all 13 features from the executed transaction sequence. The feasibility gap is measured as the relative error $|\hat{x}_i - x_i^*|/x_i^*$ between the planned feature value x_i^* and the realized value $\hat{x}_i$.

Results. Across all 10 wallets, all 13 features achieved <8.3% relative gap between planned and actual values, with a mean feasibility rate of 100%. Integer-valued features (BT, BW, MA) matched exactly because they count discrete events. Continuous features showed small stochastic deviations: `response_latency_variance` had the highest max gap (8.3%), driven by the inherent randomness of simulated inter-transaction timing; `hour_entropy` had max gap 3.1% because the hour distribution is deterministic given the transaction timestamps; `behavioral_consistency` had max gap 5.7% because the CV of inter-arrival times depends on the full sequence order.

Interpretation. These results confirm that the feature-space threat translates to problem-space with high fidelity for isolated wallets. However, two caveats apply. First, the simulator does not model gas-price market dynamics (where the wallet's gas bids must compete with real traffic), which may introduce larger deviations in `gas_price_precision` and `eip1559_tip_precision` on mainnet. Second, the simulator does not model cross-wallet correlation: if an attacker deploys 1,000 sybil wallets from the same LLM template, the resulting *population* statistics may exhibit detectable homogeneity even if each individual wallet passes feature-level checks. A full testnet deployment with real HasciDB recomputation and population-level analysis remains future work.

5 Discussion

5.1 Cross-Modality Threat Assessment

The central finding is that a single LLM adversary defeats every deployed detector class simultaneously (Table 7). This is qualitatively different from prior evasion results that target one modality: our LLM-generated sybils are designed with (a) sub-threshold HasciDB indicators, (b) human-calibrated behavioral features, and (c) low RF/MA values that avoid graph clustering. The cross-modality evasion rates (94–100%) establish a new lower bound on the difficulty of airdrop Sybil detection.

To contextualize the severity: a protocol distributing $100M via airdrop to 100K eligible addresses (roughly Arbitrum's scale) would lose $30M+ if 30% of recipients are LLM sybils that evade all detection layers. The economic incentive for attackers is therefore substantial, and the marginal cost of running an LLM adversary is negligible—our full experimental pipeline (50 sybil wallets per project across 16 projects, 3 tiers) cost approximately $12 in API credits. This asymmetry between attack cost and potential reward is the core economic threat.

5.2 What Signal Survives

`behavioral_consistency` emerges as the single most robust feature: top-1 in all 10 adversarial rounds and individually the highest-AUC single feature on moderate-tier adversaries (0.762). `hour_entropy` is strongest on basic/advanced tiers (0.864–0.878) but vulnerable to targeted evasion on moderate. The top-3 features together (behavioral_consistency + hour_entropy + response_latency_variance) reach ≥0.87 AUC across all tiers, suggesting a lightweight production detector.

The reason `behavioral_consistency` is robust where other features are not deserves explanation. This feature measures the coefficient of variation (CV) of inter-arrival times for a wallet's transactions. Human wallets exhibit high CV (irregular spacing driven by external life events), while automated agents exhibit low CV (regular intervals driven by cron jobs or event listeners). An LLM can be instructed to generate a *target* CV value that matches the human mean, but generating a realistic *distribution* of inter-arrival times—with the right skewness, kurtosis, and autocorrelation structure—requires modeling temporal dependencies that exceed what a single JSON-output prompt can achieve. In contrast, features like `hour_entropy` depend only on the marginal hour distribution (a 24-bin histogram), which is easier for the LLM to target.

5.3 The Cross-Axis Floor

Cross-axis LOPO mean AUC 0.617 is our honest floor for HasciDB. Any future detector that beats this floor without peeking at the label derivation is making real progress. Any detector reporting AUC >0.95 on HasciDB's `is_sybil` with the 5 indicators as features is measuring rule recall, not generalization.

The per-project variance (0.257–0.859, Table 5) is itself a finding: it reveals that HasciDB's labeling methodology produces internally consistent labels within each project-axis combination but does not guarantee cross-axis transferability. This has implications for any research using HasciDB as ground truth: reported AUCs should always specify whether evaluation is cross-axis or same-axis, as same-axis AUCs are inflated by circular label dependency.

5.4 Implications for Airdrop Design

Our results carry concrete implications for how protocol designers should structure eligibility criteria and anti-Sybil defenses.

1. Publish labels, not rules. HasciDB's primary value is the labeled dataset, not the detection thresholds. The moment thresholds are published, an LLM adversary can read them and plan below them (100% evasion by construction). We recommend that future airdrop audits publish labeled address sets (with provenance metadata) but keep the detection logic private until after distribution. This follows the principle from Carlini & Wagner [6]: security-through-obscurity is fragile, but forcing the attacker to guess the exact decision boundary does raise the bar.

2. Invest in execution-layer behavioral signals. Gas precision, nonce regularity, and hour entropy are cheap to compute (requiring only the transaction history from any block explorer API) and do not require off-chain identity. Our ablation shows that top-3 behavioral features alone achieve AUC $\geq$ 0.87 across all adversary tiers—a strong starting point for production deployment. Protocol teams should compute these features for all eligible addresses and flag outliers for manual review, even if they pass rule-based checks.

3. Treat detection as adversarial from day one. Pre-airdrop detection achieves AUC 0.793 on non-adversarial wallets [30] but collapses to 0% detection under LLM attack. Any production deployment must include adversarial training: generate synthetic adversarial wallets using the methodology in §3.2, augment the training set, and retrain. Our equilibrium analysis (§4.7) shows that even a single round of adversarial augmentation raises detection AUC from 0.0 (rule-based) to 0.985, and three rounds stabilize it above 0.70.

4. Combine modalities. No single detector class survives the LLM adversary: rules collapse by construction, ML detectors trained on the same features are circular, and graph detectors miss wallets with low fund-flow footprints. A layered defense combining behavioral features (execution-layer), structural features (graph topology), and temporal features (activity cadence) is necessary. The key insight is that these modalities are *complementary*: behavioral features catch LLM sybils that evade graph detectors (because they lack funding clusters), while graph detectors catch multi-account clusters that evade behavioral features (because they may exhibit human-like timing).

5.5 Toward Robust Multi-Modal Detection

The cross-modality evasion result (Table 7) suggests that no single-modality detector is sufficient. We sketch a multi-modal defense architecture that would combine the strengths of each approach.

Layer 1: Behavioral gating. Apply the top-3 AI feature detector (hour_entropy, behavioral_consistency, response_latency_variance) as a first-pass filter. Addresses scoring above a conservative threshold ($\geq$ 0.8 predicted probability) are flagged. This layer catches LLM sybils with AUC $\geq$ 0.87 and runs in $O(n)$ time per address.

Layer 2: Graph clustering. For addresses passing Layer 1, construct a transfer graph and run Louvain community detection. Flag communities with sybil-indicator fraction > 0.4. This layer catches multi-account clusters with precision 0.75 (Table 8).

Layer 3: Temporal anomaly detection. Compute temporal features (burst frequency, inter-transaction time distribution, day-of-week entropy) and flag addresses with anomalously regular patterns. This layer addresses the "too clean" signature of advanced LLM sybils.

Ensemble. Combine the three layers via a calibrated ensemble (e.g., stacking with a logistic regression meta-learner). The key assumption is that an adversary that evades all three layers simultaneously would need to: (a) match human behavioral distributions across 8 features, (b) avoid funding-flow clustering, and (c) exhibit realistically irregular temporal patterns—a conjunction of constraints that substantially raises the attack cost.

Quantitative estimate. Under independence, an adversary that evades each layer with probability p_1, p_2, p_3 evades the ensemble with probability $p_1 \cdot p_2 \cdot p_3$. Taking conservative estimates from our results: $p_1 \approx 0.40$ (behavioral at equilibrium AUC 0.60), $p_2 \approx 0.94$ (graph, lowest tier), $p_3 \approx 0.50$ (temporal, assumed), the joint evasion probability is $0.40 \times 0.94 \times 0.50 \approx 0.19$. In practice, independence is a strong assumption—the LLM can jointly optimize across layers—but even with correlation $\rho = 0.5$, the ensemble reduces evasion substantially below any single layer. Validating this multi-modal defense against an LLM adversary that jointly targets all three layers is the highest-priority item for future work.

5.6 Stackelberg Equilibrium and Formal Game-Theoretic Framing

The 10-round adversarial equilibrium can be formalized as a Stackelberg game [20, 23] between a defender (leader) and an attacker (follower):

$$\min_{\theta} \max_{\phi} \mathbb{E}_{x \sim p_\phi} \left[\mathbf{1}[f_\theta(x) < \tau] \right] \quad (2)$$
$$\text{s.t.} \quad x_i \in [\ell_i, u_i] \ \forall i \in \{1, \ldots, 13\}$$
$$\text{BT}(x) < 5, \ \text{BW}(x) < 10, \ \text{HF}(x) < 0.8$$

where f_θ is the detector parameterized by θ, p_ϕ is the LLM-generated sybil distribution parameterized by ϕ (the human-blend ratio), τ is the detection threshold, and $[\ell_i, u_i]$ are the domain constraints on each feature.

The defender moves first by choosing θ (training the detector); the attacker observes θ and responds by choosing ϕ to maximize evasion. In our protocol, the attacker's strategy space is parameterized by the human-blend ratio (0.45 to 0.97), which controls how closely the generated features match the human distribution.

The convergence to AUC ≈ 0.60 with ΔAUC < 0.01 at round 9 suggests an approximate equilibrium. At this point, the defender cannot improve without access to new feature modalities (the attacker has exhausted the blend-ratio axis), and the attacker cannot further degrade detection without violating domain constraints (blend ratio is already at 0.97—very close to the human distribution).

The equilibrium AUC of 0.60 is above chance (0.50) because `behavioral_consistency` encodes a structural invariant—the coefficient of variation of inter-arrival times—that an LLM cannot perfectly replicate without real-time traffic modeling. This residual signal represents the *cost of mimicry*: even a maximally sophisticated LLM adversary leaves a detectable footprint in the temporal structure of transactions.

5.7 Ethical Considerations

This work raises dual-use concerns that we address directly.

Responsible disclosure. We shared our evasion results with the HasciDB team prior to public release (March 2026), giving them 60 days to incorporate adversarial training into their pipeline. We do not release the full LLM prompt used for sybil generation; we release only the prompt template with the distribution-calibration context block redacted. The cached LLM responses (JSON feature vectors) are released for reproducibility, as they do not contain actionable attack code—they are numerical vectors that require the full HasciDB pipeline to be useful.

Attacker advantage is already public. The key ingredients for the attack—HasciDB's thresholds, indicator definitions, and the general idea of LLM-assisted blockchain interaction—are already publicly available. Our contribution is not to enable the attack but to *measure its effectiveness* and *develop defenses*. Without rigorous evaluation, protocol teams would continue deploying detectors with a false sense of security.

Defensive value exceeds offensive risk. The behavioral defense (AUC 0.987) and the adversarial training protocol (§4.7) are the primary deliverables of this work. A protocol team that reads this paper and deploys the top-3 feature detector with adversarial training will be substantially better defended than one that relies on HasciDB rules alone.

Broader implications. The LLM-as-adversary paradigm extends beyond airdrops. Any system where detection relies on behavioral heuristics with published thresholds is vulnerable to the same attack. Anti-fraud systems, spam filters, and bot detectors all face analogous threats as LLMs become capable of planning within numerical constraint spaces. We urge the security community to adopt adversarial evaluation as a standard practice for any threshold-based detection system.

5.8 Comparison with Traditional Adversarial ML

Our setting differs from classical adversarial ML [13, 19] in three important ways that inform defense strategies.

Low-dimensional feature space. Classical adversarial examples operate in high-dimensional spaces (e.g., $d = 784$ for MNIST, $d \sim 10^6$ for ImageNet), where small perturbations are imperceptible. Our feature space has only 13 dimensions (5 HasciDB + 8 AI), meaning the attacker must make large, semantically meaningful changes to each feature. This should, in principle, make defense easier—but the low dimensionality also means the attacker has fewer constraints to satisfy simultaneously. Our results show that even with 13 joint constraints, the LLM achieves near-perfect evasion, suggesting that the bottleneck for defense is not the number of features but their *information content* about the true generative process.

Feature-space vs. problem-space gap. In malware detection [22], the problem-space mapping $\phi : z \to \mathbf{x}$ is complex and constraining—modifying a binary's opcode sequence to change feature values while preserving functionality is hard. In airdrop detection, the mapping is simpler: the attacker directly controls the transaction sequence, and features are straightforward aggregates (counts, means, entropies). This narrower problem-space gap means that feature-space evasion rates are likely close to achievable problem-space rates, as our pilot (§4.9) confirms.

Adaptive attacker with planning capacity. Classical adversarial attacks use gradient-based optimization (PGD, C&W) to find perturbations. Our attacker uses an LLM, which brings qualitatively different capabilities: it can read natural-language descriptions of the defense, reason about inter-feature correlations, and generate diverse attack strategies without gradient access. This "zero-order" attack via natural-language planning is a new threat model that existing robustness certifications [19] do not address.

Table 15 summarizes the comparison across these three axes. The key takeaway is that airdrop Sybil detection occupies a uniquely challenging position in the adversarial ML landscape: the feature space is low-dimensional (making defense hard), the problem-space gap is narrow (making attacks feasible), and the attacker has natural-language planning capacity (making gradient-free evasion possible).

6 Limitations and Future Work

We identify seven limitations that bound the generalizability of our results, and discuss what findings would change under relaxed assumptions.

1. Feature-space attack, not problem-space. The LLM outputs a JSON feature vector, not real on-chain transactions. While our pilot (§4.9) shows <8.3% relative gap between planned and simulated feature values, a full problem-space attack would require: funding each wallet from a unique source (to avoid RF clustering), spacing transactions realistically (to match `hour_entropy` targets),

Table 15: Comparison of adversarial settings: airdrop Sybil detection vs. classical adversarial ML domains.

Property	Image Classif.	Malware Det.	Airdrop Sybil
Feature dim.	$\sim 10^6$	$\sim 10^3$	13
Problem gap	Large	Moderate	Small
Attacker	Gradient	Gradient	LLM (zero-order)
Constraint	ℓ_p ball	Functionality	Domain rules
Defense eval.	Cert. robust.	AV evasion	Cross-modality

and paying real gas costs. The gas cost constraint is non-trivial: at current Ethereum L1 gas prices (~30 Gwei, April 2026), generating a realistic 200-transaction history costs approximately 0.1–0.5 ETH per wallet (~$200–$1,000), which may be economical for high-value airdrops (Arbitrum distributed $1.2B) but prohibitive at scale for smaller campaigns. A full testnet deployment with real HasciDB recomputation remains the critical next step.

2. Proxy graph, not full transfer graph. Our Louvain evaluation uses a funding-flow proxy graph constructed from HasciDB's RF and MA indicators, not the raw Ethereum transfer graph. The proxy graph contains 8,000–10,000 nodes and 12,000–21,000 edges per project, whereas the full transfer graph may have 10×–100× more edges (including all ERC-20 transfers, DEX swaps, and bridge transactions). Real graph detectors—SybilRank [5], SybilBelief [12], and GNN-based approaches [26]—operate on this richer graph and likely achieve higher recall on real Sybils. Consequently, our LLM evasion rates (94–100%) should be interpreted as upper bounds; a full-graph evaluation may reveal structural signatures that the LLM cannot plan around without actually executing transactions.

3. Single chain (Ethereum L1 only). All 16 projects in our evaluation are Ethereum L1 airdrops. L2 rollups (Arbitrum, Optimism, Base, zkSync) differ in three ways that affect AI feature transferability: (a) gas mechanics are fundamentally different (L2 gas is 10–100× cheaper, potentially masking gas-precision signals), (b) transaction throughput is higher (enabling more realistic burst behavior without high cost), and (c) sequencer ordering introduces different timing characteristics that may alter `hour_entropy` distributions. We hypothesize that behavioral features remain informative on L2 but that their relative importance and thresholds would shift; an L2 replication study is needed.

4. Single LLM model (Claude Opus 4.6 only). Our adversary uses a single commercial LLM. Different models (GPT-4, Gemini Ultra, open-source alternatives) may produce different evasion profiles. In particular, open-source models can be fine-tuned specifically for Sybil generation, potentially achieving higher evasion rates on behavioral features. Conversely, smaller models may lack the planning capacity to jointly satisfy all 13 feature constraints, resulting in lower evasion on the advanced tier. A multi-model adversary benchmark is needed to establish the robustness of our findings across LLM families.

5. Convergence not formally reached. The adversarial equilibrium achieves ΔAUC = 0.009 at round 9 but does not reach 3 consecutive rounds below 0.01 (our pre-registered convergence criterion). The mean ΔAUC over the last 3 rounds is 0.012, suggesting convergence within 1–2 additional rounds. The equilibrium AUC of ≈0.60 should therefore be treated as an estimate; the true equilibrium may be slightly lower (0.58–0.60).

6. 4-project leakage-fix subset. The label-leakage analysis (§4.8) uses a 4-project subset (1inch, uniswap, ens, blur_s2). The full 16-project honest rerun is pending; we expect qualitatively identical results because the leakage mechanism (label-conditional sampling in `augment_with_ai_features`) is project-independent.

7. What would change our conclusions. Our central claim—that LLM adversaries defeat all deployed detector modalities—would be falsified if: (a) a full-graph detector achieves ≥50% recall on LLM sybils (indicating that graph structure captures signals invisible to indicators); (b) problem-space deployment reveals that maintaining `behavioral_consistency` < 0.2 is infeasible at scale (due to gas-cost or timing constraints); or (c) a multi-modal detector combining behavioral + graph + temporal features achieves AUC ≥ 0.95 on the adversarial equilibrium task. Any of these would narrow (though not eliminate) the threat we document.

Future work. We identify five directions: (a) Closed-loop adversarial training where the LLM generator receives reward signals from the defender, enabling RL-style policy optimization rather than static blend-ratio scheduling. (b) Problem-space attacks on an Ethereum testnet with full HasciDB recomputation, measuring the actual gas cost of evasion. (c) Cross-method transfer at scale using ≥3 independent label sources (HasciDB, Gitcoin FDD, Hop Protocol's internal Sybil list) to establish detection floors that are robust to label methodology. (d) L2 replication across Arbitrum, Optimism, and Base, testing whether behavioral features transfer across gas-pricing regimes. (e) A joint feature+graph adversary that simultaneously plans both the behavioral profile and the funding structure of each wallet, testing whether the multi-modal defense architecture sketched in §5 is robust to coordinated attacks.

7 Conclusion

We asked whether airdrop Sybil detection survives an LLM adversary. The answer is nuanced.

No single detector modality survives. LLM-generated sybils evade rules (100%), ML (100%), GBM (99.9%), and graph detection (94–100%)—the first cross-modality evasion result in this domain. The attack cost is minimal ($12 in API credits for the full benchmark), while the potential reward is enormous ($30M+ for a major airdrop), creating an asymmetry that will only grow as LLMs become cheaper and more capable.

But behavioral fingerprinting works. An 8-feature execution-layer detector achieves AUC 0.987 on the hardest adversary tier, with `behavioral_consistency` as the most robust single feature. The top-3 features by Cohen's d achieve ≥0.87 AUC across all tiers at 37.5% of the feature computation cost, providing a Pareto-optimal detector for production deployment.

The arms race converges. A 10-round Stackelberg game drives AUC from 0.985 to 0.599, with `behavioral_consistency` ranking top-1 in all rounds and feature importance distributions flattening from Gini 0.58 to 0.21. The equilibrium AUC of 0.60 is the price of using feature-space defenses alone; breaking through requires

new modalities (graph structure, temporal anomaly detection) or problem-space constraints that raise the cost of mimicry.

Methodological contributions. We documented a label-leakage bug in our own pipeline (AUC 0.953→0.609 after fix), formalized a 3-step synthetic-feature audit as Algorithm 1, and demonstrated that cross-axis evaluation is necessary to avoid the "rule-recall trap" where models trivially reconstruct labels from the features used to define them. We release code, cached LLM responses, and all experiment artifacts for reproducibility.

This benchmark establishes that LLM-driven Sybil evasion is a real and cross-modal threat, but also that behavioral invariants—particularly inter-arrival time consistency—provide a foundation for the next generation of defenses. The broader lesson extends beyond airdrops: any detection system that publishes its decision boundary in closed form is vulnerable to LLM-driven evasion, and adversarial evaluation should become standard practice.

References

[1] Giovanni Apruzzese, Pavel Laskov, Edgardo Montes de Oca, Wissam Mallouli, Luis Brdalo Rapa, Athanasios Vasileios Grammatopoulos, and Fabio Di Franco. 2023. The Role of Machine Learning in Cybersecurity. *Digital Threats: Research and Practice* 4, 1 (2023), 1–38.

[2] Arbitrum Foundation. 2023. Sybil Detection using Louvain Community Detection on Transfer Graphs. https://github.com/ArbitrumFoundation/sybil-detection.

[3] Battista Biggio and Fabio Roli. 2018. Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning. *Pattern Recognition* 84 (2018), 317–331.

[4] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.

[5] Qiang Cao, Michael Sirivianos, Xiaowei Yang, and Tiago Pregueiro. 2012. Aiding the Detection of Fake Accounts in Large Scale Social Online Services. In *Proc. NSDI*.

[6] Nicholas Carlini and David Wagner. 2017. Towards Evaluating the Robustness of Neural Networks. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 39–57.

[7] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2023. Jailbreaking Black Box Large Language Models in Twenty Queries. *arXiv preprint arXiv:2310.08419* (2023).

[8] Jacob Cohen. 1988. Statistical Power Analysis for the Behavioral Sciences. *Lawrence Erlbaum Associates* (1988).

[9] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PENTESTGPT: An LLM-empowered Automatic Penetration Testing Tool. In *33rd USENIX Security Symposium*.

[10] John R. Douceur. 2002. The Sybil Attack. In *International workshop on Peer-to-Peer Systems (IPTPS)*. 251–260.

[11] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. 2024. LLM Agents can Autonomously Hack Websites. In *arXiv preprint arXiv:2402.06664*.

[12] Neil Zhenqiang Gong, Mario Frank, and Prateek Mittal. 2014. SybilBelief: A Semi-Supervised Learning Approach for Structure-Based Sybil Detection. In *IEEE Transactions on Information Forensics and Security*, Vol. 9. 976–987.

[13] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and Harnessing Adversarial Examples. *arXiv preprint arXiv:1412.6572* (2014).

[14] Will Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, Vol. 30.

[15] Fredrik Heiding, Bruce Schneier, Arun Vishwanath, Jeremy Bernstein, and Peter S Park. 2024. Devising and Detecting Phishing Emails Using Large Language Models. *IEEE Access* 12 (2024).

[16] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30.

[17] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. 2017. Adversarial Examples in the Physical World. In *International Conference on Learning Representations (Workshop)*.

[18] Yang Li, Xiaoyu Chen, and Wei Cai. 2026. HasciDB: A Consensus-Driven Sybil Detection Framework for Cryptocurrency Airdrops. In *Proc. 2026 CHI Conference on Human Factors in Computing Systems*. ACM.

[19] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *International Conference on Learning Representations*.

[20] Martin J. Osborne and Ariel Rubinstein. 1994. A Course in Game Theory. In *MIT Press*.

[21] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. Red Teaming Language Models with Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

[22] Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. 2020. Intriguing Properties of Adversarial ML Attacks in the Problem Space. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE.

[23] Milind Tambe. 2011. Stackelberg Security Games: Looking Beyond a Decade of Success. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned* (2011).

[24] Florian Tramer, Nicholas Carlini, Wieland Brendel, and Aleksander Madry. 2020. On Adaptive Attacks to Adversarial Example Defenses. In *Advances in Neural Information Processing Systems*, Vol. 33.

[25] TrustaLabs. 2023. Airdrop Sybil Identification via Graph Community Detection. https://github.com/TrustaLabs/Airdrop-Sybil-Identification.

[26] UW Decentralized Computing Lab. 2024. ARTEMIS: Graph Neural Network-Based Sybil Detection for Blur Season 2. https://github.com/UW-Decentralized-Computing-Lab/Blur.

[27] UW Decentralized Computing Lab. 2024. LLMhunter: Multi-Expert LLM Pipeline for Sybil Classification. https://github.com/UW-Decentralized-Computing-Lab/Blur.

[28] Bimal Viswanath, Ansley Post, Krishna P. Gummadi, and Alan Mislove. 2010. An Analysis of Social Network-Based Sybil Defenses. In *Proc. ACM SIGCOMM*.

[29] Adeline Wen and collaborators. 2026. On-Chain AI Agent Identification via Execution-Layer Features. *Working Paper* (2026).

[30] Adeline Wen and collaborators. 2026. Pre-Airdrop Sybil Detection via Temporal Behavioral Features. *Working Paper* (2026).

[31] Haifeng Yu, Michael Kaminsky, Phillip B. Gibbons, and Abraham Flaxman. 2008. SybilGuard: Defending Against Sybil Attacks via Social Networks. *IEEE/ACM Transactions on Networking* 16, 3 (2008), 576–589.